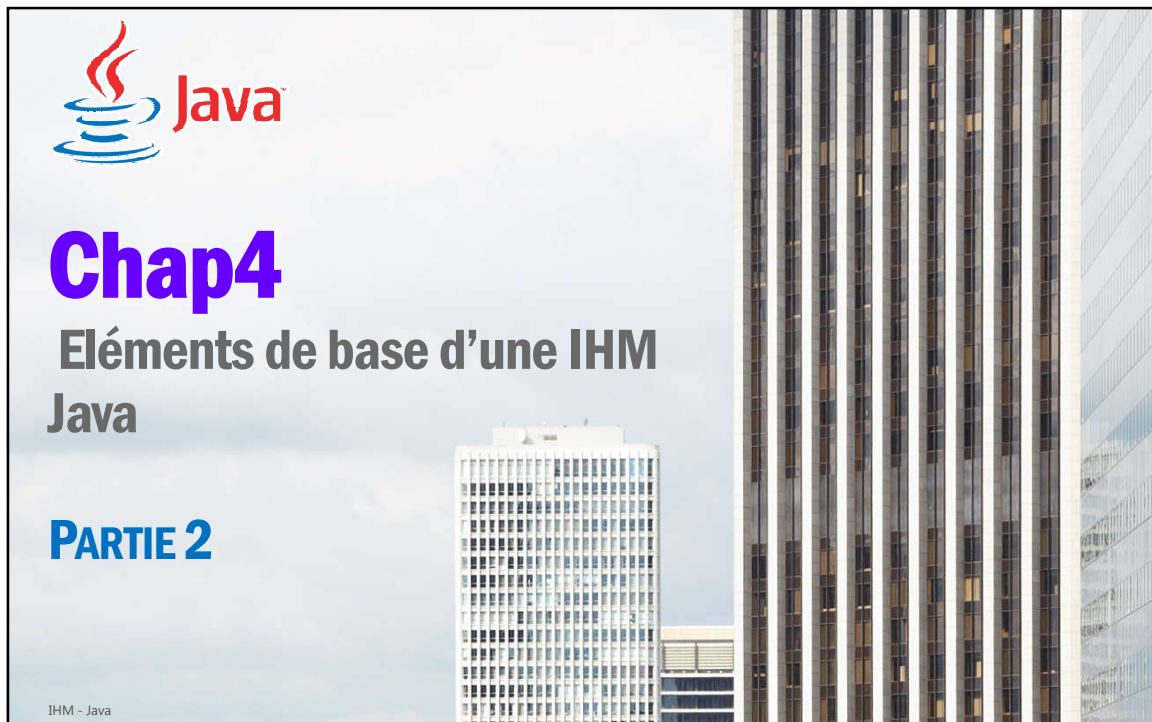




Plan du cours



Chap 1 : Initiation au Langage Java

- Introduction
- Outils de développement
- Données, Types et Operateurs
- Les structures alternatives et répétitives
- Les tableaux de données et chaînes de caractères
- Les instructions d'écriture et de lecture

Chap 2 : Java Orienté Objet

- Classes et Objets
- Héritage
- Packages
- Classes internes et Interfaces
- Les exceptions

Chap 3 : Introduction aux IHMs

- Evolution des IHM dans le temps
- Ergonomie
- Le patron d'architecture logicielle MVC

Chap 4 : Eléments de base d'une IHM Java

- Composants de base
- Composants SWING
- Les conteneurs
- Les composants atomiques
- Evènements & Ecouteurs
- Principes programmation événementielle
- Listeners java
- Gestionnaires de positionnement

Chap 5 : Connexion à une base de données

IHM - Java

2

Éléments de base d'une IHM Java

(CHAPITRE 4)



La programmation évènementielle :

- Le programme est conçu comme un ensemble d'objets qui agissent sur eux-mêmes et/ou leur environnement en réponse à des évènements provenant d'eux-mêmes et/ou de leur environnement.
- Environnement du programme: tous les systèmes qui peuvent interagir avec le programme
 - d'autres programmes
 - des matériels (imprimante)
 - des utilisateurs

Exemples

- Clic de l'utilisateur sur un bouton (environnement → objet)
- Mise à jour zone de texte (objet → objet)

Éléments de base d'une IHM Java

(CHAPITRE 4)



Les événements et les écouteurs en java :

- Un évènement « **event** » = objet qui contient toute l'information nécessaire pour que les objets et systèmes à l'écoute de l'émetteur de l'évènement puissent réagir correctement
- Exemple d'événements :
 - Clic souris
 - Déplacement de la souris
 - Case cochée
 - Fermeture de la fenêtre
- L'évènement ainsi envoyé doit être reçu et traité, C'est la fonction de l'écouteur «**listener**» appelé aussi gestionnaire d'événements ou auditeur
- Un écouteur est un objet qui contient du code qui est exécuté lorsque il reçoit l'évènement
- Un écouteur A ne peut réagir à un évènement émis par un objet B que si A est «à l'écoute» de B

Éléments de base d'une IHM Java

(CHAPITRE 4)



Les événements et les écouteurs en java :

- Les événements et les écouteurs sont des instances de classes Java dont les noms se terminent respectivement par **Event** et **Listener** : **xxxEvent** et **xxxxListener**
- En JAVA :
 - Tout évènement dérive de la classe **java.util.EventObject**
 - Tout écouteur implémente l'interface **java.util.EventListener**

Exemple

- **WindowEvent**
- **WindowListener**
- **MouseEvent**
- **MouseListener**

Éléments de base d'une IHM Java

(CHAPITRE 4)



Les événements et les écouteurs en java :

- En général et par convention, l'écouteur d'évènement d'un composant, est souvent le conteneur de ce composant. Un objet écouteur (par exemple un **JPanel**) contient un à plusieurs objets source d'évènements (par exemple des **JButton**) :
 1. L'écouteur s'enregistre auprès des sources d'évènements afin de pouvoir les écouter (Le **JPanel** doit implémenter l'interface **ActionListener** possédant une seule méthode **actionPerformed()**)
 2. Lorsqu'un évènement se produit (clic de souris), l'écouteur reçoit un objet **ActionEvent**.
 3. La méthode **getSource()** (héritée de **EventObject** superclasse des classes évènements) permet alors de déterminer quel est l'objet source de l'évènement.

Éléments de base d'une IHM Java

(CHAPITRE 4)



Les événements et les écouteurs en java :

Exemple : Gérer un Clic sur un Bouton(**JButton**)

Soit une application graphique qui permet d'afficher trois boutons, chacun permet de modifier la couleur de l'arrière plan selon la valeur de son étiquette



1. Prévoir une classe **Panneau**, de type **Jpanel**, contenant des composants et en même temps un **écouteur** des événements
2. Une autre classe Fenêtre **JFrame** où on insère notre panneau écouteur

IHM - Java

7

Éléments de base d'une IHM Java

(CHAPITRE 4)



Les événements et les écouteurs en java :

```
public class Panellistner extends JPanel implements ActionListener
{
```

```
    private JButton BoutonJaune;
    private JButton BoutonBleu;
    private JButton BoutonRouge;
    public Panellistner(){
```

```
        BoutonJaune = new JButton("Jaune");
        BoutonBleu = new JButton("Bleu");
        BoutonRouge = new JButton("Rouge");
        add(BoutonJaune);
        add(BoutonBleu);
        add(BoutonRouge);
```

```
        // Les sources d'événements sont déclarées à l'écouteur
```

```
        BoutonJaune.addActionListener(this);
        BoutonBleu.addActionListener(this);
        BoutonRouge.addActionListener(this);
```

```
    }
```

```
    //pas encore terminé : implementation de actionPerformed(ActionEvent evt) →
```

IHM - Java

8

Éléments de base d'une IHM Java

(CHAPITRE 4)



Les événements et les écouteurs en java :

```
public void actionPerformed(ActionEvent evt)
// Permet de traiter l'événement en fonction de l'objet source
{
    Object source = evt.getSource();
    Color color = getBackground();
    if (source == BoutonJaune)
        color = Color.yellow;
    else if (source == BoutonBleu)
        color = Color.blue;
    else if (source == BoutonRouge)
        color = Color.red;
    setBackground(color);
}

// je suis toujours dans la classe Panellistner

}
```

IHM - Java

9

Éléments de base d'une IHM Java

(CHAPITRE 4)



Les événements et les écouteurs en java :

```
class Fenetre extends JFrame
{
    public Fenetre()
    {
        setTitle("ButtonTest");
        setSize(300, 200);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setContentPane(new Panellistner());
        setVisible(true);
    }
}

public class Test2
{
    public static void main(String[] args)
    {
        new Fenetre();
    }
}
```

IHM - Java

10

Éléments de base d'une IHM Java

(CHAPITRE 4)



Les événements et les écouteurs en java :

Autre alternative : Ecouteur

On peut ajouter directement l'écouteur sur le bouton cible sans l'ajouter sur le panneau :

```
public class Panellistner extends JPanel{
    private JButton BoutonJaune;
    ...
    public Panellistner(){
        BoutonJaune = new JButton("Jaune");
        ...
        add(BoutonJaune);
        ...
        BoutonJaune.addActionListener(new ActionListener(){
            public void actionPerformed(ActionEvent evt){
                setBackground(Color.yellow);
            }
        });
    }
}
```

Inconvénient : Même traitement pour chaque composant interactif

IHM - Java

11

Éléments de base d'une IHM Java

(CHAPITRE 4)



Les événements et les écouteurs en java :

Autre alternative : Ecouteur

```
class Fenetre extends JFrame {

    public Fenetre() {
        setTitle("ButtonTest");
        setSize(300, 200);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setContentPane(new Panellistner());
        setVisible(true);
    }

    public class Test {
        public static void main(String[] args) {
            new Fenetre();
        }
    }
}
```

IHM - Java

12

Éléments de base d'une IHM Java

(CHAPITRE 4)



```
class Fenetre extends JFrame {
```

```
    public Fenetre() {
```

```
        setTitle("ButtonTest");
```

```
        setSize(300, 200);
```

```
        setDefaultCloseOperation(EXIT_ON_CLOSE);
```

```
        setContentPane(new Panellistner());
```

```
        setVisible(true);    }
```

```
    }
```

```
class Fenetre extends JFrame {
```

```
    public Fenetre() {
```

```
        setTitle("ButtonTest");
```

```
        setSize(300, 200);
```

```
        addWindowListener(
            new WindowAdapter() {
                public void windowClosing(
                    WindowEvent e) {
                    System.exit(0);
                }
            }
        );
```

```
        setContentPane(
```

```
            new Panellistner());
```

```
        setVisible(true);    }
```

```
    }
```

IHM - Java

13

Éléments de base d'une IHM Java

(CHAPITRE 4)



Les interfaces écouteurs d'évènements :

- Une classe qui désire recevoir des événements doit implémenter une interface de type écouteur
- Elle se recense auprès de la source d'événement, puis elle reçoit les événements souhaités et les traite grâce aux méthodes de l'interface écouteur
- Java dispose des deux packages suivants :
 - **java.awt.event**
 - **java.swing.event**

IHM - Java

14

Éléments de base d'une IHM Java

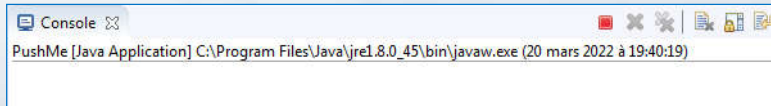
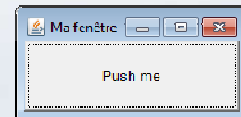
(CHAPITRE 4)



Exemple Simple:

```
public class PushMe implements ActionListener {
    // Le constructeur de la classe PushMe
    public PushMe() {
        JFrame f = new JFrame("Ma fenêtre");
        JButton b = new JButton("Push me");
        f.add(b); // On ajoute le bouton dans la fenêtre
        f.setVisible(true);
    }
    // gestionnaire d'évènement défini dans ActionListener
    public void actionPerformed(ActionEvent evt) {
        System.out.println("Bouton " + evt.getSource() + " cliqué !");
    }
}

static public void main(String argv[]) {
    new PushMe();
}
```



IHM - Java

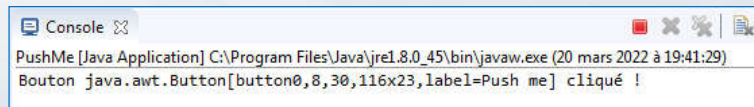
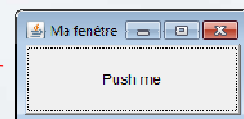
Éléments de base d'une IHM Java

(CHAPITRE 4)



Exemple Simple:

```
public class PushMe implements ActionListener {
    // Le constructeur de la classe PushMe
    public PushMe() {
        JFrame f = new JFrame("Ma fenêtre");
        JButton b = new JButton("Push me");
        b.addActionListener(this);
        f.add(b); // On ajoute le bouton dans la fenêtre
        f.pack();
        f.setVisible(true);
    }
    // gestionnaire d'évènement défini dans ActionListener
    public void actionPerformed(ActionEvent event) {
        System.out.println("Bouton " + event.getSource() + " cliqué !");
    }
}
```



IHM - Java

Éléments de base d'une IHM Java

(CHAPITRE 4)



Les adaptateurs :

- Certaines interfaces écouteurs sont accompagnées d'adaptateurs.
- Ils implémentent toutes les méthodes de l'interface afin qu'elles n'accomplissent aucune action.
- En effet, toutes les méthodes d'une interface ne sont pas forcément utilisées.
 - l'adaptateur évite d'avoir à les citer dans le programme
- Ces adaptateurs sont les suivants :
 1. **ComponentAdapter**
 2. **ContainerAdapter**
 3. **FocusAdapter**
 4. **KeyAdapter**
 5. **MouseAdapter**
 6. **MouseMotionAdapter**
 7. **WindowAdapter**

IHM - Java

17

Éléments de base d'une IHM Java

(CHAPITRE 4)



```
public class PushMe implements ActionListener {
    public PushMe() {
        JFrame f = new JFrame("Ma fenêtre");
        JButton b = new JButton("Push me");
        b.addActionListener(this);

        f.addWindowListener(new WindowAdapter() {
            public void windowClosing(WindowEvent event) {
                System.exit(0);
            }
        });

        f.add(b); f.pack();
        f.setVisible(true);
    }
    // gestionnaire d'évènement définit dans ActionListener
    public void actionPerformed(ActionEvent event) {
        System.out.println("Bouton " + event.getSource() + " cliqué !");
    }
    static public void main(String argv[]) {
        new PushMe();
    }
}
```

IHM - Java

18

Éléments de base d'une IHM Java

(CHAPITRE 4)



le package `java.awt.event` :

Il y a onze interfaces écouteurs dans le package `java.awt.event`

1. ActionListener // clic, touche entrée, sélection d'un élément

- Méthode : `actionPerformed(ActionEvent e)`
- Événements générés par : `AbstractButton`, `Button`, `JComboBox`, `JFileChooser`, `TextField`, `List`, `MenuItem`, `TextField` avec la méthode `addActionListener`

2. AdjustmentListener // déplacement du curseur d'une barre de défilement

- Méthode : `adjustmentValueChanged(AdjustmentEvent e)`
- Événements générés par : `Scrollbar`, `JScrollbar` avec la méthode `addAdjustmentListener`

Éléments de base d'une IHM Java

(CHAPITRE 4)



3. ComponentListener // déplacement, affichage, masquage ou modification de taille de composants

- Méthodes : `componentHidden(ComponentEvent e)`
`componentMoved(ComponentEvent e)`
`componentResized(ComponentEvent e)`
`componentShown(ComponentEvent e)`
- Événements générés par : `Component` avec la méthode `addComponentListener`

4. ContainerListener // ajout ou suppression d'un composant dans un conteneur

- Méthodes : `componentAdded(ContainerEvent e)`
`componentRemoved(ContainerEvent e)`
- Événements générés par : `Container` avec la méthode `addContainerListener`

5. FocusListener // obtention ou perte du focus par un composant

- Méthodes : `focusGained(FocusEvent e)`
`focusLost(FocusEvent e)`
- Événements générés par : `Component` avec la méthode `addFocusListener`

Éléments de base d'une IHM Java

(CHAPITRE 4)



6. ItemListener //sélection dans une liste ou dans un groupe de cases à cocher

- Méthodes : `itemStateChanged(ItemEvent e)`
- Événements générés par : `AbstractButton`, `Checkbox`, `CheckboxMenuItem`, `Choice`, `JComboBox`

7. ListKeyListener //action sur une touche du clavier (pressée ou relâchée)

- Méthodes : `keyPressed(KeyEvent e)`
`keyReleased(KeyEvent e)`
`keyTyped(KeyEvent e)`
- Événements générés par : `Component` avec la méthode `addKeyListener`

8. MouseListener //clic sur bouton, déplacement du pointeur

- Méthodes : `mouseClicked(MouseEvent e)`
`mouseEntered(MouseEvent e)`
`mouseExited(MouseEvent e)`
`mousePressed(MouseEvent e)`
`mouseReleased(MouseEvent e)`
- Événements générés par : `Component` avec la méthode `addMouseListener`

IHM - Java

21

Éléments de base d'une IHM Java

(CHAPITRE 4)



9. MouseMotionListener //événements de glisser-déplacé

- Méthodes : `mouseDragged(MouseEvent e)`
`mouseMoved(MouseEvent e)`
- Événements générés par : `Component` avec la méthode `addMouseMotionListener`

10. TextListener //modification du texte d'un composant texte

- Méthodes : `textValueChanged(TextEvent e)`
- Événements générés par : `TextComponent` avec la méthode `addTextListener`

11. WindowListener //fenêtre activée, désactivée, réduite, fermée, ...

- Méthodes : `windowActivated(WindowEvent e)`
`windowClosed(WindowEvent e)`
`windowClosing(WindowEvent e)`
`windowDeactivated(WindowEvent e)`
`windowDeiconified(WindowEvent e)`
`windowIconified(WindowEvent e)`
`windowOpened(WindowEvent e)`
- Événements générés par : `Window` avec la méthode `addWindowListener`

IHM - Java

22

Éléments de base d'une IHM Java

(CHAPITRE 4)



le package java.swing.event :

1. [AncestorListener](#)

2. [CaretListener](#)

- Méthode : `caretUpdate(CaretEvent e)`
- Événements générés par : `JTextComponent` avec la méthode `addCaretListener`

3. [CellEditorListener](#)

4. [ChangeListener](#)

- Méthode : `stateChanged(ChangeEvent e)`
- Événements générés par : `AbstractButton`, `JProgressBar`, `JViewport` avec la méthode `addChangeListener`

5. [DocumentListener](#)

6. [HyperlinkListener](#)

7. [InternalFrameListener](#)

8. [ListDataListener](#)

9. [ListSelectionListener](#)

- Méthode : `valueChanged(ListSelectionEvent e)`
- Événements générés par : `JList` avec la méthode `addListSelectionListener`

IHM - Java

23

Éléments de base d'une IHM Java

(CHAPITRE 4)



le package java.swing.event :

10. [MenuDragMouseListener](#)

- Méthodes : `menuDragMouseDragged(MenuDragMouseEvent e)`
`menuDragMouseEntered(MenuDragMouseEvent e)`
`menuDragMouseExited(MenuDragMouseEvent e)`
`menuDragMouseReleased(MenuDragMouseEvent e)`
- Événements générés par : `JMenuItem` avec la méthode `addMenuDragMouseListener`

11. [MenuKeyListener](#)

- Méthodes : `menuKeyPressed(MenuKeyEvent e)`
`menuKeyReleased(MenuKeyEvent e)`
`menuKeyTyped(MenuKeyEvent e)`
- Événements générés par : `JMenuItem` avec la méthode `addMenuKeyListener`

12. [MenuListener](#)

- Méthodes : `menuCanceled(MenuEvent e)`
`menuDeselected(MenuEvent e)`
`menuSelected(MenuEvent e)`
- Événements générés par : `JMenu` avec la méthode `addMenuListener`

IHM - Java

24

Éléments de base d'une IHM Java

(CHAPITRE 4)



le package `java.swing.event` :

13. [MouseListener](#)

14. [PopupMenuListener](#)

- Méthodes : `popupMenuCanceled(PopupMenuEvent e)`
`popupMenuWillBecomeInvisible(PopupMenuEvent e)`
`popupMenuWillBecomeVisible(PopupMenuEvent e)`
- Événements générés par : `JPopupMenu` avec la méthode `addPopupMenuListener`

15. [TableColumnModelListener](#)

16. [TableModelListener](#)

17. [TreeExpansionListener](#)

18. [TreeModelListener](#)

19. [TreeSelectionListener](#)

20. [TreeWillExpandListener](#)

21. [UndoableEditListener](#)

IHM - Java

25

Éléments de base d'une IHM Java

(CHAPITRE 4)



Etude d'un cas :

Nous reprenons l'exemple de TP N°4, on souhaite ajouter des écouteurs pour les composants `nom`, `prenom` et `adresse` dans l'onglet `Identite`.

IHM - Java

26

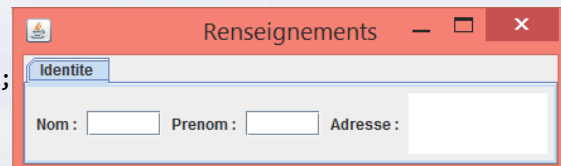
Éléments de base d'une IHM Java

(CHAPITRE 4)



Etude d'un cas :

```
public class FicheIdentite extends JPanel{
    private JLabel labelNom ;    private JTextField nom ;
    private JTextField prenom ; private JLabel labelPrenom;
    private JTextArea adresse ; private JLabel labelAdresse;
    public FicheIdentite() {
        labelNom = new JLabel ( " Nom : " ) ;
        labelPrenom = new JLabel ( " Prenom : " ) ;
        labelAdresse = new JLabel ( " Adresse : " ) ;
        nom = new JTextField(5) ;
        prenom = new JTextField(5) ;
        adresse = new JTextArea("",3,10) ;
        this.add(labelNom);this.add(nom);
        this.add(labelPrenom);this.add(prenom) ;
        this.add(labelAdresse);this.add(adresse);
    }
}
```



IHM - Java

27

Éléments de base d'une IHM Java

(CHAPITRE 4)



Etude d'un cas :

Solution 1 Ajouter à chaque composant un écouteur

- Les composants n'ont pas forcément les mêmes écouteurs.
- **ActionListener** n'est pas un écouteur de **TextArea**, nous devons utiliser un écouteur adéquat pour ce composant et implanter ses méthodes
- Pour **TextArea** nous choisissons **KeyListener**

IHM - Java

28

Éléments de base d'une IHM Java

(CHAPITRE 4)



```
public class FicheIdentite extends JPanel implements ActionListener, KeyListener{
    private JLabel labelNom ;    private JTextField nom ;
    private JTextField prenom ; private JLabel labelPrenom;
    private JTextArea adresse ; private JLabel labelAdresse;

    public FicheIdentite() {
        super () ;
        labelNom = new JLabel ( " Nom : " ) ; nom = new JTextField(5) ;
        labelPrenom = new JLabel ( " Prenom : " ) ; prenom = new JTextField(5) ;
        labelAdresse = new JLabel ( " Adresse :" ) ; adresse = new JTextArea("",3,10) ;
        this.add(labelNom);this.add(nom);nom.addActionListener(this);
        this.add(labelPrenom);this.add(prenom );prenom.addActionListener(this);
        this.add(labelAdresse);this.add(adresse);adresse.addKeyListener(this);}

    public void actionPerformed(ActionEvent evt){
        if(nom.getText().equals("")||prenom.getText().equals(""))
            JOptionPane.showMessageDialog(null,"Remplir ce champ");}

    public void keyTyped(KeyEvent arg0) {}
    public void keyReleased(KeyEvent arg0) {}
    public void keyPressed(KeyEvent arg0) {if(adresse.getText().equals(""))
        JOptionPane.showMessageDialog(null, "Remplir ce champ"); }
```

IHM - Java }

29

Éléments de base d'une IHM Java

(CHAPITRE 4)



Etude d'un cas :

Solution2 : En utilisant l'adaptateur KeyAdapter

```
public class FicheIdentite extends JPanel implements ActionListener{
    private JLabel labelNom ;    private JTextField nom ;
    private JTextField prenom ; private JLabel labelPrenom;
    private JTextArea adresse ; private JLabel labelAdresse;
    public FicheIdentite() {
        //...
        adresse.addKeyListener(new KeyAdapter(){
            public void keyPressed(KeyEvent arg0) {
                if (adresse.getText().equals(""))
                    JOptionPane.showMessageDialog(null, "Remplir ce champ"); }
            });
    }
    public void actionPerformed(ActionEvent evt){
        if(nom.getText().equals("")||prenom.getText().equals(""))
            JOptionPane.showMessageDialog(null,"Remplir ce champ");}
    }
```

IHM - Java

30

Éléments de base d'une IHM Java

(CHAPITRE 4)



Les gestionnaires de disposition :

- Un gestionnaire de disposition ou « **Layout Manager** » est un objet permettant de disposer au mieux les composants dans un conteneur
- Le programmeur se contente de modéliser le contenu du conteneur en utilisant les termes d'un layout manager existant
- Lors de la création d'un conteneur, un gestionnaire par défaut est créé en même temps et lui est associé.
- 4 principaux conteneurs

Conteneur	Gestionnaire par défaut
JPanel, JApplet	FlowLayout
JFrame, JWindow	BorderLayout

- Un même conteneur ne peut posséder qu'un seul layout manager

IHM - Java

31

Éléments de base d'une IHM Java

(CHAPITRE 4)



Les gestionnaires de disposition :

- Chaque layout manager implémente l'interface **java.awt.LayoutManager**
- Il utilise les méthodes **getPreferredSize/getMinimumSize**, pour connaître la taille préférée/minimale des composants.
- Il calcule les tailles et les positions des composants en utilisant les méthodes **setSize** et **setLocation**.
- Pour accéder au gestionnaire de placement d'un conteneur on utilise la méthode **getLayout**. La méthode **setLayout** permet de spécifier un nouveau gestionnaire.

Exemple de LayoutManager :

```
FlowLayout
GridLayout
ViewportLayout
ScrollPaneLayout
BorderLayout
BoxLayout
```

IHM - Java

32

Éléments de base d'une IHM Java

(CHAPITRE 4)



FlowLayout :

Le gestionnaire le plus élémentaire. Il implémente le concept d'une séquence horizontale dont les paramètres essentiels sont :

- L'alignement (à droite, à gauche, centré,...) de bas en haut
- L'espacement horizontal séparant les composants
- L'espacement vertical séparant les lignes

Les justifications sont définies à l'aide de variables statiques

FlowLayout.LEFT, FlowLayout.CENTER et FlowLayout.RIGHT

Éléments de base d'une IHM Java

(CHAPITRE 4)



FlowLayout :

```
class PanelFlowLayout extends JPanel {
    public PanelFlowLayout() {
        FlowLayout fl = new FlowLayout();
        this.setLayout(fl);
        this.add(new JButton ("Un"));
        this.add(new JButton (" Deux"));
        this.add(new JButton (" Trois"));
        this.add(new JButton (" Quatre "));
        this.add(new JButton (" Cinq"));
        this.add(new JButton (" Six"));
    }
}
```



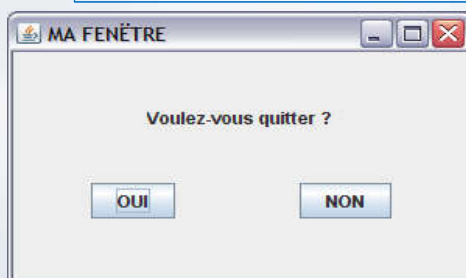
Éléments de base d'une IHM Java

(CHAPITRE 4)



FlowLayout :

```
...
FlowLayout f1 = new FlowLayout(FlowLayout.CENTER, 80, 40);
this.setLayout(f1);
this.add(new JLabel("Voulez-vous quitter ?"));
this.add(new JButton (" Oui"));
this.add(new JButton (" Non"));
...
```



- Si les dimensions du conteneur changent, le positionnement des composants est recalculé
- On peut figer les bordures :
`setResizable(false);`

IHM - Java

35

Éléments de base d'une IHM Java

(CHAPITRE 4)



GridLayout :

Implémente le concept d'une disposition en tableau (grille) dont les paramètres essentiels sont

- Le nombre de lignes
- Le nombre de colonnes
- L'espace entre les colonnes
- L'espace entre les lignes

Généralement les composants sont disposés de gauche à droite puis de haut en bas

IHM - Java

36

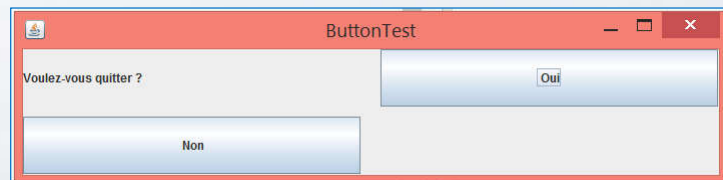
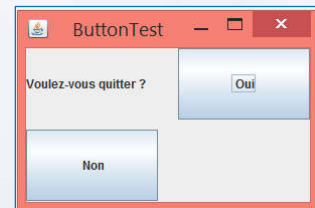
Eléments de base d'une IHM Java

(CHAPITRE 4)



GridLayout :

```
GridLayout gl = new GridLayout(2,2,20,10);
this.setLayout(gl);
this.add(new JLabel("Voulez-vous quitter ?"));
this.add(new JButton (" Oui"));
this.add(new JButton (" Non"));
```



IHM - Java

37

Eléments de base d'une IHM Java

(CHAPITRE 4)



GridLayout :

```
class PanelGridLayout extends JPanel {
public PanelGridLayout () {
    GridLayout gl = new GridLayout(3,2);//(3,0)
    this.setLayout(gl);
    this.add (new JButton (" Deux"));
    this.add (new JButton (" Trois"));
    this.add (new JButton (" Quatre "));
    this.add (new JButton (" Cinq"));
    this.add (new JButton (" Six"));
    this.add (new JButton (" Sept"));
}
}
```



IHM - Java

38

Eléments de base d'une IHM Java

(CHAPITRE 4)



BorderLayout :

- Divise la zone d'affichage en cinq zones dans le conteneur : une zone centrale (**CENTER**) et quatre zones périphériques (**EAST**, **WEST**, **NORTH**, **SOUTH**).
- Ses principaux paramètres sont
 - L'espacement horizontal entre les composants
 - L'espacement vertical entre les composants
- Il n'est pas obligatoire de placer cinq composants dans le conteneur, les emplacements non pourvu seront alors ignorés

IHM - Java

39

Eléments de base d'une IHM Java

(CHAPITRE 4)



BorderLayout :

```
class PanelBorderLayout extends JPanel {
    public PanelBorderLayout() {
        BorderLayout bl = new BorderLayout();
        this.setLayout(bl);
        this.add (new JButton (" Deux"), BorderLayout. NORTH);
        this.add (new JButton (" Trois"), BorderLayout. SOUTH);
        this.add (new JButton (" Quatre "), BorderLayout. EAST);
        this.add (new JButton (" Cinq"), BorderLayout. WEST);
    }
}
```



IHM - Java

40

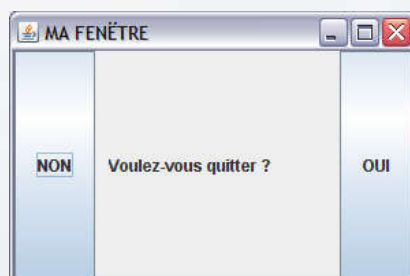
Éléments de base d'une IHM Java

(CHAPITRE 4)



BorderLayout :

```
setLayout(new BorderLayout(10,40));  
add("Center",new JLabel("Voulez-vous quitter ?"));  
add("East",new JButton("OUI"));  
add("West",new JButton("NON"));
```



IHM - Java

41

Éléments de base d'une IHM Java

(CHAPITRE 4)



Rendre l'interface réellement indépendante du système :

- Certaines valeurs sont indiquées en pixels
- Le résultat obtenu sera donc différent en fonction de la **résolution** de l'écran.
- Une solution consiste à utiliser les dimensions des composants pour dimensionner la fenêtre.

IHM - Java

42

Éléments de base d'une IHM Java

(CHAPITRE 4)



Rendre l'interface réellement indépendante du système :

- Certaines valeurs sont indiquées en pixels
- Le résultat obtenu sera donc différent en fonction de la **résolution** de l'écran.
- Une solution consiste à utiliser les dimensions des composants pour dimensionner la fenêtre.

Exemple:

```
Dimension tailleEcran = Toolkit.getDefaultToolkit().getScreenSize();
int largeurEcran=tailleEcran.width;
int hauteurEcran=tailleEcran.height;
int largeur=220;
int hauteur=140;
int xPos=(largeurEcran-largeur)/2;
int yPos=(hauteurEcran-hauteur)/2;
setBounds(xPos, yPos, largeur, hauteur);
setVisible(true);
```

IHM - Java

43

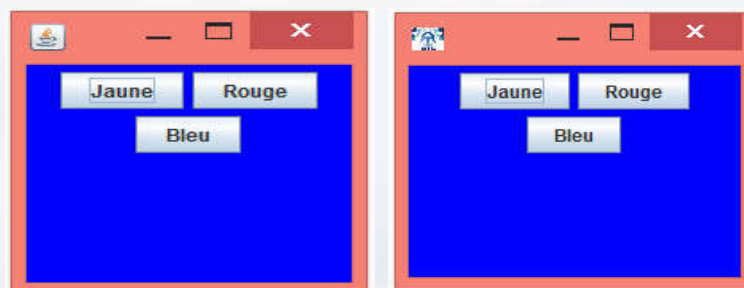
Éléments de base d'une IHM Java

(CHAPITRE 4)



Modifier l'icone de l'application :

```
Image icone = Toolkit.getDefaultToolkit().getImage("E:\\fst.png");
this.setIconImage(icone);
```



IHM - Java

44

Éléments de base d'une IHM Java

(CHAPITRE 4)



Exercice :

On désire réaliser une application **MVC** permettant de sélectionner un pays et d'afficher un ensemble d'informations relatives au pays sélectionné.

Pays/Capitale

Choisir un pays

- ☐ Maroc
- ☐ France
- ☐ Espagne
- ☐ USA
- ☐ Allemagne
- ☐ Canada
- ☐ Japon
- ☐ Chine
- ☐ Bresil

Informations

Drapeau :

Capitale :

Monnaie :

Langue :

Quitter

IHM - Java

45